

Pruebas de Performance Laboratorio

Grupo 6 - 2025

Taller de Sistemas de Información Java EE

Integrantes:

- Bibiana Fariello
- Felipe Masliah
- Franco Piroto
- Matias Bidarte
- Paola Benedicetti
- Braian Romero

Tutor: Ing. Emiliano Matinez

Tabla de contenido

1. Introducción.....	2
1.1. Objetivo del Documento.....	3
2. Objetos de análisis.....	4
3. Ambiente de pruebas.....	5
4. Herramientas.....	5
5. Pruebas realizadas.....	5
5.1. Prueba 1.....	6
5.2. Prueba 2.....	6
5.3. Prueba 3.....	7
5.4. Prueba 4.....	8
5.5. Prueba 5.....	9
5.6. Prueba 6.....	10
6. Conclusiones.....	12

1. Introducción

1.1. Objetivo del Documento

El objetivo principal de estas pruebas de estrés es identificar los límites de capacidad del sistema, evaluando cómo se comporta la aplicación bajo cargas crecientes de usuarios simultáneos. Se busca detectar:

- El punto a partir del cual comienzan a aparecer errores significativos.
- La relación entre la velocidad de incremento de usuarios (ramp-up) y el rendimiento del sistema.
- La estabilidad y capacidad de respuesta del backend frente a escenarios extremos.

Esta información permite ajustar la infraestructura, optimizar el código y definir parámetros seguros para el entorno de producción. Cada prueba implica un total de **19.590 requests** ejecutados contra los endpoints configurados.

2. Objetos de análisis

Las pruebas se realizaron sobre los siguientes servicios REST.

Validar conexión:

GET: */api/validar*

GET: */api/pruebas/consultarMayoriaEdad?cedula={cedula}*

GET: */api/validar-conexion-clinicas*

GET: */api/validar-conexion-prestadores*

Autenticación con GubUy:

GET: */api/autenticacionGubuy/solicitarLogin?appToken={token}*

GET: */api/autenticacionGubuy/logout?state={state}&plataforma={plat}*

GET: */api/resumenDigital/solicitar?nroDocumento={cedula}*

Sesión:

GET: */api/sesion/isActive?token={token}*

GET: */api/sesion/close?token={token}*

Alta de usuarios:

POST: */api/usuario/crearOActualizar*

Documentos clínicos:

POST: */api/documentos-clinicos/crearDocumento*

GET: */api/documentos-clinicos/listar*

GET: */api/documentos-clinicos/{id}?cedulaMedico={cedula}*

Solicitud de acceso

GET: */api/solicitudes-acceso/pendientes?tipoDocumento={td}&nroDocumento={cedula}*

POST:

/api/solicitudes-acceso?tipoDocumentoPaciente=\${tipoDocPaciente}&nroDocumentoPaciente=\${nroDocPaciente}&tipoDocumentoProfesional=\${tipoDocProfesional}&nroDocumentoProfesional=\${nroDocProfesional}&tenantId=\${tenantId}

POST: */api/solicitudes-acceso/\${idSolicitud}/aprobar*

POST: */api/solicitudes-acceso/\${idSolicitud}/rechazar*

3. Ambiente de pruebas

Con el fin de validar la performance real del sistema, las pruebas se realizan sobre el ambiente de producción, alojado en Elastic Cloud. Además, la base de datos se encuentra alojada en supabase.

Las características de estos ambientes se detallan a continuación.

Contenedor Wildfly 37.0.1:

- 3 CPU, 6.4GHZ
- 2GiB RAM
- 200GB disco
- 60 IOPS
- Configuración de Wildfly con standalone.xml optimizado.

Contenedor de Base de datos (Supabase):

- CPU compartido
- 0.5 GB RAM
- 43mbps Disk Throughput
- 250 IOPS
- 60 Database max connections

4. Herramientas

Las herramientas utilizadas para realizar la simulación y medir la actividad son las siguientes:

- Apache JMeter 5.6.3.
- Gráficas de uso de recursos de la plataforma Elastic Cloud.

5. Pruebas realizadas.

Esta sección detalla las pruebas realizadas.

5.1. Prueba 1

En esta primera prueba se busca establecer una línea base del rendimiento del sistema, bajo una carga inicial controlada.

Con **1500 usuarios** y un **ramp-up de 1500 segundos**, se simuló un ingreso paulatino de **1 usuario por segundo**. Cada usuario ejecutó solicitudes hacia todos los endpoints hasta completar los 1500 requests por endpoint.

Esta prueba permite medir el comportamiento del servidor en un escenario estable, con baja presión y ritmo constante, ideal para detectar latencias iniciales, identificar cuellos de botella leves y validar que las configuraciones básicas del entorno estén correctas.

Datos de prueba:

Usuarios: 1500

Ramp-Up: 1500 segundos (1 usuario por segundo)

Total de requests: 19590

Tiempos:

Hora de comienzo: 5:06 AM

Hora de finalización: 5:32 AM

Duración: 25:26 minutos

Resultados:

Resultado: 12.9 requests/segundo

Tasa de error: 0.01%



5.2. Prueba 2

En esta segunda prueba, se busca incrementar ligeramente la carga reduciendo el ramp-up a 1200 segundos, equivalente a un 20% de reducción respecto a la prueba anterior.

Esto genera que los usuarios ingresen más rápido al sistema (1 usuario cada 0.80 segundos).

El objetivo principal es medir cómo responde el sistema ante un aumento moderado en la velocidad de incorporación de usuarios, y validar si esto provoca un impacto significativo en el throughput o en el porcentaje de errores registrados

Datos de prueba:

Usuarios: 1500

Ramp-Up: 1200 segundos (1 usuario cada 0.80 segundos)

Total de requests: 19590

Tiempos:

Hora de comienzo: 5:44 AM

Hora de finalización: 6:05 AM

Duración: 20:27 minutos

Resultados:

Resultado: 16.1 requests/segundo

Tasa de error: 0.03%



5.3. Prueba 3

En esta instancia se aplica un cambio más agresivo: se reduce el ramp-up a 600 segundos, lo que implica duplicar la presión respecto a la prueba 2.

Ahora, los usuarios ingresan cada 0.4 segundos.

Esta prueba permite observar cómo reacciona el sistema ante un escenario de mayor concurrencia en menor tiempo, evaluando tanto la estabilidad como la capacidad de sostener el incremento del throughput sin degradarse en exceso.

Datos de prueba:

Usuarios: 1500

Ramp-Up: 600 segundos (1 usuario cada 0.40 segundos)

Total de requests: 19590

Tiempos:

Hora de comienzo: 6:19 AM

Hora de finalización: 6:29 AM

Duración: 10:28 minutos

Resultados:

Resultado: 31.7 requests/segundo

Tasa de error: 2.54%



5.4. Prueba 4

Aquí se llevó al sistema a una situación de alta exigencia reduciendo el ramp-up a solo 150 segundos. Esto implica que los 1500 usuarios ingresaron en apenas 2 minutos y medio, presionando con 1 usuario cada 0.1 segundos. El objetivo fue observar el punto de quiebre del sistema, determinando niveles críticos de saturación y las tasas de error generadas cuando la demanda supera la capacidad real de procesamiento.

Datos de prueba:

Usuarios: 1500

Ramp-Up: 150 segundos (1 usuario cada 0.10 segundos)

Total de requests: 19590

Tiempos:

Hora de comienzo: 6:47 AM

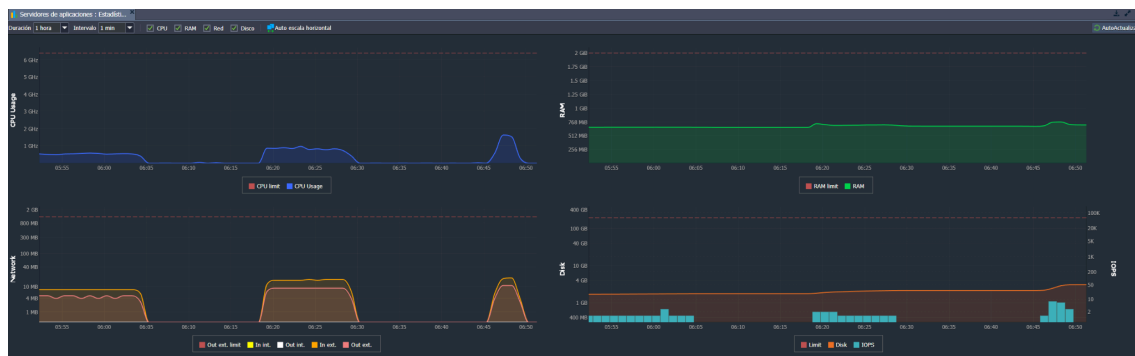
Hora de finalización: 6:50 AM

Duración: 2:57 minutos

Resultados:

Resultado: 117 requests/segundo

Tasa de error: 66.25%



5.5. Prueba 5

Luego de encontrar un nivel de saturación elevado en la prueba 4, se comienza a incrementar nuevamente el ramp-up, con el fin de buscar un punto de equilibrio entre rendimiento y estabilidad.

Con un ramp-up de 300 segundos (equivalente al 80% de la prueba 1, y 50% de la prueba 3), se pretende verificar si al disminuir la presión instantánea el sistema logra reducir la tasa de error, sin sacrificar demasiado throughput.

Esta prueba aporta información clave sobre la recuperación del sistema al reducir el estrés abrupto.

Datos de prueba:

Usuarios: 1500

Ramp-Up: 300 segundos (1 usuario cada 0.20 segundos)

Total de requests: 19590

Tiempos:

Hora de comienzo: 7:01 AM

Hora de finalización: 7:06 AM

Duración: 5:27 minutos

Resultados:

Resultado: 61.6 requests/segundo

Tasa de error: 39.84%



5.6. Prueba 6

En esta prueba se ajusta el ramp-up a 450 segundos. Representa un aumento del 200% respecto a la prueba 4, reduciendo notablemente el estrés inicial.

El propósito es encontrar un balance entre alta carga y baja tasa de error, identificando un rango más estable de rendimiento donde el sistema pueda sostener un volumen importante de operaciones, sin degradarse rápidamente.

Datos de prueba:

Usuarios: 1500

Ramp-Up: 450 segundos (1 usuario cada 0.30 segundos)

Total de requests: 19590

Tiempos:

Hora de comienzo: 7:13 AM

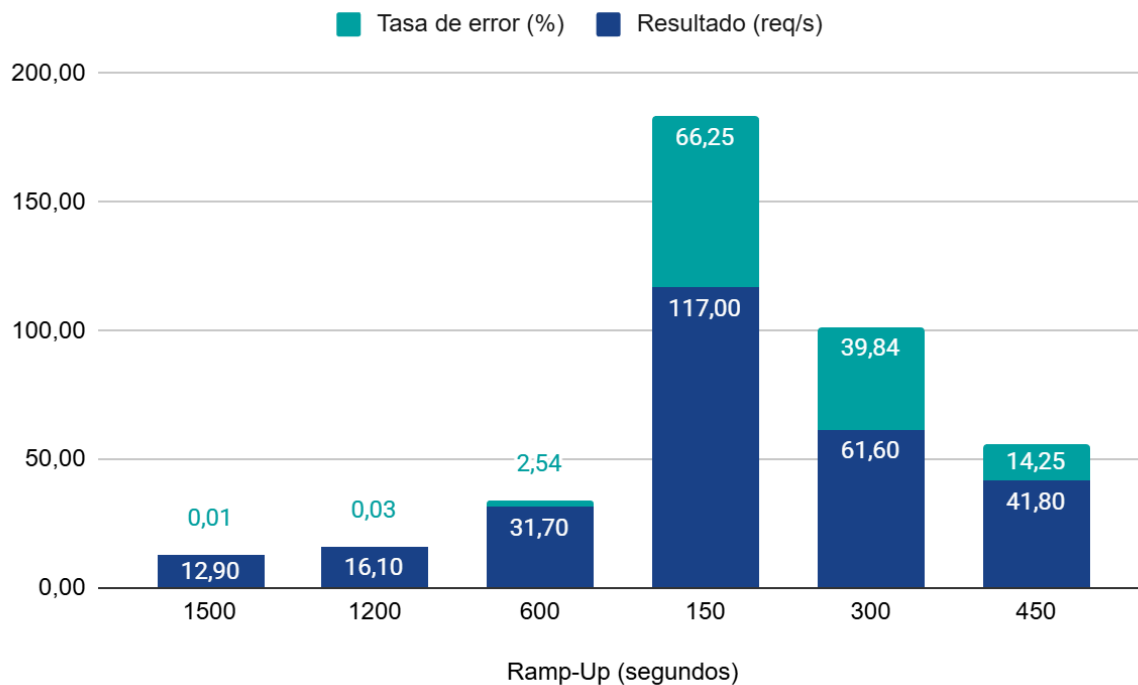
Hora de finalización: 7:21 AM

Duración: 7:58 minutos

Resultados:

Resultado: 41.8 requests/segundo

Tasa de error: 14.25%



6. Conclusiones

Los errores registrados en las diferentes pruebas, guardan relación directa con las limitaciones del entorno de base de datos utilizado.

En todos los casos, el error visualizado es:

```
{“message”:”Unable to acquire JDBC Connection [jakarta.resource.ResourceException: IJ000453:
Unable to get managed connection for java:/jdbc/PostgresDS] [n/a]”,”success”:false}
```

El servicio de hosting de base de datos (Supabase), opera bajo un plan gratuito que establece un máximo de **60 conexiones simultáneas** en el pool. Debido a esta restricción, al incrementar la concurrencia y reducir el ramp-up, el sistema alcanza rápidamente el límite de conexiones disponibles, generando así fallos en la ejecución de las llamadas.

En consecuencia, los porcentajes de error observados no reflejan únicamente la capacidad real de la aplicación, sino también la saturación del motor de base de datos bajo las condiciones impuestas por el plan gratuito.

Para obtener resultados más precisos y representar la capacidad real del sistema en producción, sería recomendable ejecutar pruebas en un entorno con mayor capacidad de conexiones o con parámetros ajustados que eviten el agotamiento del pool.